TECHNICAL REPORT

Implementation of Real-Time Wrist Roll Control for Dual UR3e Robotic Arms via Leap Motion Sensor

Shubham J. Sonara
Mechatronics Lab, University of New Haven, West Haven, CT, USA
Date: May 9, 2026

1. Introduction

This report documents the implementation of real-time wrist roll motion control added to the gesture-based dual robotic arm teleoperation system.The base system used a Leap Motion Controller 2 to map hand position and limited orientation data to two UR3e collaborative robotic arms via the RTDE protocol.

The original system identified wrist mobility and orientation control as a key limitation, noting that only two orientation axes — roll and yaw — were partially mapped, leaving the third axis fixed. The implementation described in this report directly addresses that limitation by adding continuous, real-time palm roll tracking and mapping it to each robot arm's end-effector rotation, allowing the operator to rotate the robot's tool by physically rotating their wrist.

2. Background and Motivation

In the base system, each UR3e arm's end-effector was controlled primarily through 3D position mapping: hand movements in X, Y, and Z were translated to corresponding robot movements scaled by a factor $k \approx 1.5 \times 10^{-3}$. End-effector orientation was not continuously tracked — the gripper maintained a fixed approach angle unless recalibrated.

For practical tasks such as pick-and-place, assembly, or object manipulation, the ability to rotate the robot's gripper is essential. A screw cannot be driven, a cup cannot be tilted, and an asymmetric object cannot be correctly oriented without end-effector rotation. The Leap Motion SDK provides rich hand orientation data, including roll (rotation of the palm about the forward axis), which is a natural and intuitive motion for a human operator. Adding this capability directly responds to user feedback gathered during testing of the original system, where users expressed the desire to rotate objects using wrist twists.

3. Implementation Details

3.1 Roll Angle Computation

The palm roll angle is extracted from the Leap Motion hand data using the palm's surface normal vector. The normal vector (nx, ny, nz) points perpendicular to the palm surface. The roll angle is computed using the arctangent of the x and y components of this normal:

```
def get_hand_roll(hand):
    n = hand.palm.normal
    return np.arctan2(n.x, n.y)
```

This yields a signed angle in radians representing the tilt of the palm about its forward axis. When the palm faces downward flat, this value is near zero. Rotating the wrist clockwise or counter-clockwise produces proportional positive or negative values. The function is wrapped in a try/except block to handle cases where tracking data may be momentarily unavailable, returning 0.0 as a safe default.

3.2 Roll Calibration

To avoid discontinuities and arbitrary starting orientations, the roll angle is captured at the moment of calibration. The user initiates calibration by performing a strong pinch gesture with their hand. At that moment, the system records both the hand's 3D position and its roll angle as a baseline reference:

```
def calibrate(self, leap_pos, leap_roll=0.0):
    self.robot_calib_pose = self.rtde_r.getActualTCPPose()
    self.calibration_offset = np.array([leap_pos.x, leap_pos.y,
leap_pos.z])
    self.calibration_roll = leap_roll
    self.calibrated = True
```

All subsequent roll values are computed as a delta relative to this baseline. This means the robot's gripper remains at its calibration orientation when the user's wrist is in its natural calibration posture — only deliberate wrist rotation produces robot movement.

3.3 Roll-to-Rotation Mapping

During normal teleoperation, the roll delta is computed in each control cycle and mapped to an end-effector rotation. The transformation is applied on top of the calibrated robot pose:

```
roll_delta = leap_roll - self.calibration_roll
angle = clamp(roll_delta * self.ROTATE_SCALE,
              -self.max_tool_rotation,
               self.max_tool_rotation)
```

The clamping ensures the robot never exceeds ±90° of tool rotation from the calibration pose, preventing dangerous over-rotation or joint limit violations. The ROTATE_SCALE parameter controls sensitivity and is tuned differently for each arm:

- Left arm: ROTATE_SCALE = 0.6 (higher sensitivity, more responsive)
- Right arm: ROTATE_SCALE = 0.35 (lower sensitivity, more controlled)

This asymmetry was introduced after testing showed that the right arm's workspace geometry made it more prone to abrupt motion at higher sensitivities, requiring a more conservative setting for safe operation.

3.4 Rotation Axis Assignment

The roll rotation is applied about different axes depending on which arm is being controlled, to match the physical geometry of each arm's mounting:

```
axis = 'x' if self.is_right else 'z'
```

The left arm rotates about the Z-axis of its tool frame, while the right arm rotates about its X-axis. This choice was determined empirically by observing which axis produced the most intuitive correspondence between the operator's wrist rotation and the gripper's visible rotation for each arm's configuration on the test bench.

3.5 Rotation Composition

Rather than setting an absolute rotation angle, the roll is composed with the calibration pose's existing rotation using quaternion-based multiplication (implemented via scipy's Rotation class). This preserves the arm's calibrated orientation and applies roll on top of it:

```
current_rot = R.from_rotvec(self.robot_calib_pose[3:6])
tool_roll = R.from_euler(axis, angle, degrees=False)
new_rot = current_rot * tool_roll
new_rotvec = new_rot.as_rotvec()
```

The resulting rotation vector is combined with the target position and sent to the robot as a 6-element pose [x, y, z, rx, ry, rz]. Using rotation composition (rather than direct angle assignment) prevents gimbal lock and ensures smooth, continuous rotation across the full ±90° range.

3.6 Motion Smoothing

To prevent jerky rotation from hand tremors or rapid wrist movements, the full pose (including roll) is smoothed via exponential interpolation in the robot control loop:

```
alpha = 0.08 if self.is_right else 0.12
self.command_pose = [c + alpha * (t - c)
                     for c, t in zip(self.command_pose, self.target_pose)]
```

The alpha parameter (0.08 for the right arm, 0.12 for the left) controls how quickly the command pose follows the target. Lower alpha values produce smoother but slightly lagging motion; higher values respond faster but may amplify small hand tremors. These values were tuned during testing to balance responsiveness and stability.

4. Configuration Summary

The following table summarizes the key parameters implemented for roll control across both arms:

Parameter	Left Arm	Right Arm
Rotation Sensitivity (ROTATE_SCALE)	0.6	0.35
Rotation Axis	Z-axis	X-axis
Max Tool Rotation	±90°	±90°
Interpolation Factor (alpha)	0.12	0.08
Roll Computed From	Palm Normal (arctan2)	Palm Normal (arctan2)
Calibration Roll Captured	Yes (pinch gesture)	Yes (pinch gesture)

5. Integration with the Existing System

The roll control feature was integrated into the existing MultiHandTracker and UR3eController classes without modifying the overall architecture. The changes touched three areas:

- get_hand_roll() — a new standalone function that extracts roll from any Leap Motion hand object.
- calibrate() — extended to accept and store the hand's roll angle at calibration time.
- transform_position() — extended to compute roll delta, clamp it, compose it with the calibration rotation, and return the full 6-DOF target pose including orientation.

The robot control loop itself was not modified; it already accepted and forwarded 6-element poses via servoL(). The hand dispatch in on_tracking_event() was likewise unchanged — hand_roll is extracted and passed alongside hand position on every tracking event.

6. Challenges and Design Decisions

6.1 Choosing the Correct Rotation Axis

The most significant implementation challenge was determining which local tool frame axis to rotate about for each arm. Rotating about the wrong axis produces unintuitive motion (e.g., the gripper pitching forward instead of rolling in-place). This required iterative testing with the physical robot, observing the direction of rotation for each

candidate axis (x, y, z) and selecting the one that matched the operator's expectation when rotating their wrist.

6.2 Sensitivity Tuning

The ROTATE_SCALE parameter required careful tuning. Too high a value caused the robot to reach its ±90° clamp limit with only a small wrist rotation, making fine control difficult. Too low a value required exaggerated wrist movement to achieve the desired rotation. The final values (0.6 for left, 0.35 for right) were selected after testing with multiple users to find the most natural feel for each arm's geometry.

6.3 Preventing Over-Rotation

Without clamping, large or rapid wrist rotations could command the robot into joint singularities or violate physical limits. The ±90° clamp (max_tool_rotation = np.radians(90)) was set to match the practical range of human wrist rotation while keeping the robot well within safe operating bounds. Future work could make this limit configurable per task.

6.4 Smooth Transition at Calibration

An early design issue arose when the robot would snap to a new orientation at the moment of calibration if the user's hand was not flat. Capturing the calibration roll and computing all subsequent angles as deltas relative to it solved this cleanly — the robot holds its position at calibration and only rotates when the user deliberately twists their wrist.

7. Observed Performance

After implementing roll control, the system was tested with pick-and-place and object reorientation tasks. Key observations included:

- Operators could rotate the gripper smoothly through approximately ±75° of practical range before reaching the clamp limit, covering nearly all common gripping orientations.
- The exponential smoothing effectively eliminated visible jitter from minor hand tremor during held positions, while still allowing fluid intentional rotation.
- Calibration correctly zeroed the roll reference, meaning operators with different natural wrist angles at rest all experienced consistent, neutral starting behavior.
- The asymmetric sensitivity settings (0.6 left, 0.35 right) were well-received by test users, who found the right arm's lower sensitivity helpful when making small, precise rotational adjustments.
- No unintended rotation was observed during position-only movements, confirming that the delta-based mapping correctly decoupled translation from rotation.

The addition of roll control did not noticeably increase system latency. The roll computation adds only a few microseconds per frame (two floating-point operations and a scipy rotation), well below the 50 ms control loop period.

8. Conclusion

This implementation adds continuous, real-time wrist roll control to the dual-arm teleoperation system, directly addressing the wrist mobility limitation identified in the IDETC 2025 paper. By capturing the palm normal's roll angle from the Leap Motion SDK, applying delta-based calibration, composing the rotation with the robot's calibration pose using quaternion multiplication, and smoothing the output via exponential interpolation, the system now provides intuitive one-to-one correspondence between the operator's wrist rotation and the robot gripper's rotation.

The implementation is lightweight, fully integrated into the existing codebase, and requires no additional hardware or sensors. It represents a practical step toward full 6-DOF end-effector control, with the remaining gap being pitch (approach angle) control, which is identified as the next enhancement in the ongoing research roadmap.